```
# ================================
# GEN AI EXPERIMENT
# Zero-shot vs Few-shot vs Chain-of-Thought Prompting
# Dataset: GSM8K
# Model: TinyLlama-1.1B-Chat
# Framework: PyTorch
# ================================

# STEP 1: Install required libraries
!pip install -q transformers torch accelerate pandas pyarrow datasets

# STEP 2: Import libraries
import torch
import pandas as pd
import re
from transformers import AutoTokenizer, AutoModelForCausalLM
from datasets import load_dataset

# STEP 3: Load GSM8K dataset from HuggingFace
gsm8k = load_dataset("gsm8k", "main")
# Convert to pandas DataFrame (train split)
data = pd.DataFrame(gsm8k['train'])
# Only keep 'question' and 'answer' columns
data = data[['question', 'answer']]
# Save to Parquet (optional)
data.to_parquet("gsm8k.parquet")
print("Total samples:", len(data))
print(data.head())

# STEP 4: Device configuration
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)

# STEP 5: Load TinyLlama model
model_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16 if device == "cuda" else torch.float32,
    low_cpu_mem_usage=True
).to(device)
model.eval()

# STEP 6: Build prompts
def build_prompt(question, mode="zero_shot", exemplars=None):
    if mode == "zero_shot":
        return f"Solve the following math problem.\nGive only the final numeric answer.\n\nQuestion: {question}\nAnswer: "
    elif mode == "few_shot":
        prompt = "Here are some solved examples:\n\n"
        for q, a in exemplars:
            prompt += f"Q: {q}\nA: {a}\n\n"
        prompt += f"Now solve the following problem.\nQ: {question}\nA: "
        return prompt
    elif mode == "cot":
        return f"Solve the following math problem step by step.\nAfter reasoning, give the final answer in the format:\nAnswer: 
    else:
        raise ValueError("Invalid prompting mode")

# STEP 7: Generate answer
def generate_answer(prompt, max_new_tokens=128):
    inputs = tokenizer(prompt, return_tensors="pt").to(device)
    with torch.no_grad():
        outputs = model.generate(**inputs, max_new_tokens=max_new_tokens, do_sample=False)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

# STEP 8: Extract numeric answer
def extract_last_number(text):
    numbers = re.findall(r"-?\d+\.?\d*", text)
    return numbers[-1] if numbers else None

# STEP 9: Select few-shot examples
def get_few_shot_examples(k=3):
    examples = []
    for i in range(k):
        q = data.iloc[i]["question"]
```

```
        a = data.iloc[i]["answer"].split("####")[-1].strip()
        examples.append((q, a))
    return examples

few_shot_examples = get_few_shot_examples(k=3)

# STEP 10: Evaluate a prompting mode
def evaluate_mode(mode, n_samples=50):
    correct = 0
    hallucinations = 0
    for i in range(n_samples):
        question = data.iloc[i]["question"]
        gt_answer = data.iloc[i]["answer"].split("####")[-1].strip()

        prompt = build_prompt(question, mode, few_shot_examples if mode=="few_shot" else None)
        output = generate_answer(prompt)
        pred = extract_last_number(output)
        gt = extract_last_number(gt_answer)

        if pred is None:
            hallucinations += 1
        elif pred == gt:
            correct += 1

        if (i+1) % 10 == 0:
            print(f"[{mode}] Processed {i+1}/{n_samples}")

    accuracy = correct / n_samples
    hallucination_rate = hallucinations / n_samples
    print(f"\nMode: {mode}")
    print(f"Accuracy: {accuracy:.3f}")
    print(f"Hallucination Rate: {hallucination_rate:.3f}\n")
    return accuracy, hallucination_rate

# STEP 11: Run the experiment
acc_zero, hall_zero = evaluate_mode("zero_shot")
acc_few, hall_few = evaluate_mode("few_shot")
acc_cot, hall_cot = evaluate_mode("cot")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set i
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
README.md:       7.93k/? [00:00<00:00, 214kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to e
main/train-00000-of-00001.parquet: 100%                                  2.31M/2.31M [00:00<00:00, 206kB/s]
main/test-00000-of-00001.parquet: 100%                                   419k/419k [00:00<00:00, 1.94MB/s]
Generating train split: 100%                                  7473/7473 [00:00<00:00, 8090.10 examples/s]
Generating test split: 100%                                   1319/1319 [00:00<00:00, 22446.18 examples/s]
Total samples: 7473
                                            question  \
0  Natalia sold clips to 48 of her friends in Apr...
1  Weng earns $12 an hour for babysitting. Yester...
2  Betty is saving money for a new wallet which c...
3  Julie is reading a 120-page book. Yesterday, s...
4  James writes a 3-page letter to 2 different fr...

                                              answer
0  Natalia sold 48/2 = <<48/2=24>>24 clips in May...
1  Weng earns 12/60 = $<<12/60=0.2>>0.2 per minut...
2  In the beginning, Betty has only 100 / 2 = $<<...
3  Maila read 12 x 2 = <<12*2=24>>24 pages today....
4  He writes each friend 3*2=<<3*2=6>>6 pages a w...
Using device: cuda
config.json: 100%                                     608/608 [00:00<00:00, 22.9kB/s]
tokenizer_config.json:       1.29k/? [00:00<00:00, 51.0kB/s]
tokenizer.json:       1.84M/? [00:00<00:00, 18.5MB/s]
tokenizer.model: 100%                                 500k/500k [00:00<00:00, 1.06MB/s]
special_tokens_map.json: 100%                                  551/551 [00:00<00:00, 13.1kB/s]
`torch_dtype` is deprecated! Use `dtype` instead!
model.safetensors: 100%                               2.20G/2.20G [00:20<00:00, 127MB/s]
Loading weights: 100%                                 201/201 [00:03<00:00, 35.82it/s, Materializing param=model.norm.weight]
generation_config.json: 100%                                   124/124 [00:00<00:00, 11.1kB/s]
[zero_shot] Processed 10/50
[zero_shot] Processed 20/50
[zero_shot] Processed 30/50
[zero_shot] Processed 40/50
[zero_shot] Processed 50/50

Mode: zero_shot
Accuracy: 0.000
Hallucination Rate: 0.000

[few_shot] Processed 10/50
[few shot] Processed 20/50
```